

Projection aléatoire : embeddings efficaces pour graphes hétérogènes

Présentation technique de 30 minutes

Votre Nom

Votre Institution

1^{er} juillet 2025

- 1 Définition du problème et motivation
- 2 Limites des marches aléatoires
- 3 Projection aléatoire : intuition
- 4 Fondements théoriques
- 5 Algorithme et implémentation
- 6 Optimisation et apprentissage
- 7 Comparaison avec les marches aléatoires
- 8 Résultats et performances
- 9 Résumé comparatif
- 10 Points essentiels

Notre jeu de données

- 28 702 auteurs
- 20 conférences
- 8 920 termes
- Relations multiples : Auteur–Auteur, Auteur–Conférence, Auteur–Terme

Objectif

Apprendre des embeddings de dimension 256 qui capturent la sémantique afin de prédire les collaborations futures.

Pourquoi est-ce difficile ?

- Graphe hétérogène (plusieurs types de nœuds)
- Connectivité clairsemée
- Passage à l'échelle : >28 000 nœuds
- Complexité sémantique : chaque relation a un sens différent

Algorithme classique

- Pour chaque nœud :
 - Générer des marches aléatoires : $A \rightarrow C \rightarrow A \rightarrow T \rightarrow A \rightarrow \dots$
 - Considérer les marches comme des « phrases »
 - Appliquer Word2Vec (Skip-gram) pour apprendre les embeddings

Problèmes

- **Coût computationnel** : millions de marches nécessaires
- **Mémoire** : stockage de toutes les séquences
- **Biais d'échantillonnage** : perte de la structure globale
- **Sensibilité aux hyperparamètres** : longueur et nombre de marches

Exemple : pour 28 000 auteurs, 10 marches $\times$ 100 pas = 28 M de pas !

Idée clé

- 1 **Créer une matrice aléatoire creuse** R (calcul économique)
- 2 **Appliquer directement les opérations de graphe** : MR , M^2R , M^3R
- 3 **Combiner intelligemment** les caractéristiques via des poids appris

Pourquoi cela fonctionne-t-il ?

Le lemme de Johnson–Lindenstrauss garantit que les projections aléatoires préservent les distances dans les espaces de grande dimension.

Analogie

Au lieu d'explorer une ville en marchant au hasard, on prend des photos aériennes sous différents angles puis on les combine !

Theorem (Lemme de Johnson–Lindenstrauss (1984))

Pour tout ensemble de n points dans un espace de grande dimension, il existe une projection dans $O(\log n)$ dimensions qui préserve toutes les distances paires dans $(1 \pm \varepsilon)$.

Formulation formelle

Pour des points $x_1, \dots, x_n \in \mathbb{R}^d$, il existe une matrice aléatoire $R \in \mathbb{R}^{d \times k}$, avec $k = O(\log n / \varepsilon^2)$, telle que :

$$(1 - \varepsilon) \|x_i - x_j\|^2 \leq \|R^T x_i - R^T x_j\|^2 \leq (1 + \varepsilon) \|x_i - x_j\|^2$$

Conséquences pour nous

- Les projections aléatoires préservent la structure de voisinage.
- On peut travailler dans des dimensions plus basses sans perte significative.
- Fournit une garantie théorique pour notre approche.

Étape 1 : construction de la matrice de projection

Dimensions

$R \in \mathbb{R}^{n \times d}$ (28 702 $\times$ 256 dans notre cas)

Motif de sparsité

Chaque colonne possède exactement $s = 3$ éléments non nuls :

$$R_{ij} = \begin{cases} \frac{+1}{\sqrt{s}} & \text{avec probabilité } \frac{s}{n} \\ \frac{-1}{\sqrt{s}} & \text{avec probabilité } \frac{s}{n} \\ 0 & \text{sinon} \end{cases}$$

Exemple pour 6 auteurs, 4 dimensions, $s = 2$

$$R = \begin{bmatrix} 0 & +0.7 & 0 & -0.7 \\ -0.7 & 0 & +0.7 & 0 \\ 0 & 0 & -0.7 & +0.7 \\ +0.7 & -0.7 & 0 & 0 \\ 0 & +0.7 & -0.7 & 0 \\ -0.7 & 0 & 0 & +0.7 \end{bmatrix}$$

Étape 2 : pondération par degré (α)

Problème

Les nœuds de haut degré (auteurs prolifiques) dominent les embeddings.

Solution

Pondération par degré avec $\alpha = -0.5$:

$$R' = D^\alpha R$$

Exemple avec $\alpha = -0.5$

Degrés des auteurs : [100, 10, 5, 50, 20, 8]

$$D^{-0.5} = \text{diag}([0.10, 0.32, 0.45, 0.14, 0.22, 0.35])$$

Avant pondération :

$$A_1 : [0, +0.7, 0, -0.7]$$

$$A_2 : [-0.7, 0, +0.7, 0]$$

Après pondération :

$$A_1 : [0, +0.07, 0, -0.07] \text{ (Atténué)}$$

$$A_2 : [-0.22, 0, +0.22, 0] \text{ (Amplifié)}$$

Étape 3 : matrices d'adjacence des méta-chemins

Capturer la sémantique

$$\mathbf{ACA} \text{ (Auteur} \rightsquigarrow \text{Confrence} \rightsquigarrow \text{Auteur)} : M_{ACA} = A_{AC}A_{CA} \quad (1)$$

$$\mathbf{ATA} \text{ (Auteur} \rightsquigarrow \text{Terme} \rightsquigarrow \text{Auteur)} : M_{ATA} = A_{AT}A_{TA} \quad (2)$$

$$\mathbf{AAA} \text{ (Auteur} \rightsquigarrow \text{Auteur} \rightsquigarrow \text{Auteur)} : M_{AAA} = A_{AA}A_{AA} \quad (3)$$

Exemple de dimensions

- $A_{AC} : 28\,702 \times 20$ (auteurs $\times$ conférences)
- $A_{CA} : 20 \times 28\,702$ (conférences $\times$ auteurs)
- $M_{ACA} : 28\,702 \times 28\,702$ (auteurs $\times$ auteurs)

Interprétation

$M_{ACA}[i, j]$ = nombre de conférences dans lesquelles les auteurs i et j ont co-publié.

Étape 4 : normalisation (β)

Problème

Certains auteurs ont beaucoup de connexions, d'autres très peu.

Solution

Normalisation par ligne avec $\beta = -1.0$:

$$\tilde{M} = D^{\beta} M$$

Exemple

Sommes de lignes brutes de M_{ACA} : [50, 5, 100, 20]

$$D^{-1} = \text{diag}([1/50, 1/5, 1/100, 1/20])$$

Avant normalisation : [10, 5, 30, 5] (conférences partagées)

Après normalisation : [0.2, 0.1, 0.6, 0.1] (probabilités)

Étape 5 : génération des caractéristiques

Innovation clé

Au lieu de calculer explicitement M^2 ou M^3 , on itère :

Algorithme des puissances (itératif)

- $U_0 = R'$ (matrice aléatoire pondérée)
- $U_1 = \text{normalize}(\tilde{M} U_0)$ (caractéristiques à 1 saut)
- $U_2 = \text{normalize}(\tilde{M} U_1)$ (2 sauts)
- $U_3 = \text{normalize}(\tilde{M} U_2)$ (3 sauts)

Exemple concret (méta-chemin ACA)

$U_0 = R' \in \mathbb{R}^{28,702 \times 256}$ (caractéristiques initiales)

$U_1 = \tilde{M}_{ACA} U_0$ (auteurs reliés via 1 conférence)

$U_2 = \tilde{M}_{ACA} U_1$ (via 2 conférences)

$U_3 = \tilde{M}_{ACA} U_2$ (via 3 conférences)

Efficacité mémoire

Ne jamais stocker $\tilde{M}^2$ ou $\tilde{M}^3$ ($28k \times 28k \times 8 \approx 6,4$ Go chacun !)

Étape 6 : collecte multi-échelle des caractéristiques

Pour chaque couple méta-chemin $\times$ puissance

Méta-chemins : [AAA, ACA, ATA]

Puissances : [1, 2, 3]

Nombre total de matrices : $3 \times 3 = 9$

Tenseur de caractéristiques : $F \in \mathbb{R}^{9 \times 28,702 \times 256}$

Interprétation

- $F[0]$: AAA¹ – collaborations directes
- $F[1]$: AAA² – collaborations à 2 sauts
- $F[2]$: AAA³ – collaborations à 3 sauts
- $F[3]$: ACA¹ – conférences partagées
- $F[4]$: ACA² – communautés de conférence
- $F[5]$: ACA³ – domaines de recherche élargis
- $F[6]$: ATA¹ – mots-clés partagés
- $F[7]$: ATA² – voisinages thématiques
- $F[8]$: ATA³ – domaines de recherche

Étape 7 : combinaison de caractéristiques

Problème

Quelles caractéristiques sont les plus importantes ? Laissons le modèle l'apprendre !

Poids apprenables (softmax)

$$\theta \in \mathbb{R}^9 \quad (4)$$

$$w = \text{softmax}(\theta) \quad \left(\sum_i w_i = 1 \right) \quad (5)$$

$$Z = \sum_{i=0}^8 w_i F[i] \quad (\text{embedding final}) \quad (6)$$

Exemple de poids appris

AAA ¹ : 0.05	AAA ² : 0.12	AAA ³ : 0.08
ACA¹ : 0.25	ACA² : 0.30	ACA³ : 0.15
ATA ¹ : 0.02	ATA ² : 0.02	ATA ³ : 0.01

Observation : le modèle apprend que les relations de conférence sont les plus prédictives !

Étape 8 : prédiction de lien par distance

Score basé sur la distance

Pour les auteurs i et j , on obtient les embeddings $z_i, z_j \in \mathbb{R}^{256}$:

$$\text{distance}^2 = \|z_i - z_j\|^2 = \sum_{k=1}^{256} (z_{i,k} - z_{j,k})^2 \quad (7)$$

$$\text{probabilité} = \sigma(\gamma - \lambda \text{distance}^2) \quad (8)$$

Intuition

- Petite distance $\Rightarrow$ forte probabilité de collaboration
- Grande distance $\Rightarrow$ faible probabilité

Paramètres apprenables

- γ (intercepte) : probabilité de base
- λ (pente) : influence de la distance

Deux termes

$$\mathcal{L} = \mathcal{L}_{BCE} + \lambda_{entropy} \mathcal{L}_{entropy}$$

1. Entropie croisée binaire (BCE)

$$\mathcal{L}_{BCE} = -[y \log \hat{p} + (1 - y) \log(1 - \hat{p})]$$

- Perte standard de prédiction de liens
- Pondération pour déséquilibre des classes (ratio 3 :1 négatif : positif)

2. Régularisation entropique

$$\mathcal{L}_{entropy} = - \sum_{i=1}^9 w_i \log w_i$$

- Empêche le modèle de se concentrer sur une seule caractéristique
- Encourage la diversité des caractéristiques utilisées

Aspect	Marches aléatoires (Node2Vec)	FastRP (notre méthode)
Complexité temporelle	$O(rwl\ n)$	$O(E\ d\ p)$
Mémoire	$O(rwl\ n)$	$O(d\ p)$
Prétraitement	Génération des marches	Ops matricielles creuses
Scalabilité	Faible (quadratique)	Excellente (linéaire)

Paramètres

- r : marches par nœud (10-50)
- w : longueur de marche (80-100)
- l : époques d'entraînement (≈ 100)
- E : nombre d'arêtes, d : dimension, p : puissances

Chiffres réels pour notre jeu de données

- Node2Vec : $\approx 28k \times 10 \times 80 \times 100 = 2,24$ G opérations
- FastRP : $\approx 500k$ arêtes $\times 256 \times 3 = 384$ M opérations
- **$\approx 6\times$ plus rapide !**

Pourquoi la projection aléatoire est-elle meilleure ?

1 Préservation de la structure globale

- Les marches sont locales (limitées par w)
- Les puissances matricielles capturent la connectivité globale

2 Relations multi-échelle

- Puissances 1-2-3 : immédiat, communauté, domaine
- Marches : voisinage de longueur fixe

3 Caractéristiques déterministes

- Pas de variance d'échantillonnage
- Résultats reproductibles

4 Garanties théoriques

- Lemme de Johnson–Lindenstrauss

5 Support hétérogène

- Gestion naturelle de multiples types de relations

Principaux chiffres

- **AUC entraînement** : 0,94
- **AUC validation** : 0,94
- **Temps d'entraînement** : 15 min (vs. heures pour les marches)

Importance des caractéristiques

Type de relation	Poids
Conférences (ACA)	70 %
Collaborations (AAA)	25 %
Similarité thématique (ATA)	5 %

Impact du filtrage de nœuds

- Original : 28 702 auteurs
- Filtré (degré ≥ 5) : 27 821 auteurs
- 881 nœuds « bruit » supprimés
- Amélioration du clustering

Prédire une collaboration (Alice & Bob)

Étape 1 : IDs : Alice = 1000, Bob = 1500

Étape 2 : Extraire leurs embeddings

```
z_alice = Z[1000, :]
```

```
z_bob = Z[1500, :]
```

Étape 3 : Calculer la distance

```
distance_sq = |z_alice - z_bob|**2
```

Étape 4 : Probabilité de collaboration

```
proba = sigmoid( $\gamma - \lambda \times distance\_sq$ )
```

Résultat : 41% de chance qu'Alice et Bob collaborent.

Projection aléatoire vs marches

Avantages :

- ✓ **Vitesse** : 6× plus rapide que Node2Vec
- ✓ **Mémoire** : usage constant
- ✓ **Qualité** : meilleure structure globale
- ✓ **Déterministe** : pas de variance d'échantillonnage
- ✓ **Scalable** : complexité linéaire
- ✓ **Interprétable** : poids des caractéristiques
- ✓ **Hétérogène** : support natif multi-relations

Inconvénients potentiels :

- ! **Complexité** : plus de paramètres (α , β , puissances)
- ! **Maturité théorique** : moins étudié que les marches
- ! **Implémentation** : manipulation de matrices creuses

En résumé

La projection aléatoire offre un meilleur compromis vitesse-qualité pour les graphes hétérogènes.

À retenir

1. Idée centrale

Projections aléatoires + puissances matricielles = embeddings efficaces.

2. Fondement mathématique

Lemme de Johnson–Lindenstrauss.

3. Innovation ingénierie

Calcul itératif évitant l'explosion mémoire : $M \circ (M \circ R)$ plutôt que $M^2 \circ R$.

4. Gains de performance

6× plus rapide que les marches aléatoires.

5. Impact pratique

Applicable aux grands graphes hétérogènes réels.

Vue d'ensemble

La projection aléatoire rend l'embedding de graphes pratique à grande échelle.

Code et implémentation

- Disponible sur GitHub : [Votre dépôt]
- Implémentation Python (PyTorch)
- Support CPU, CUDA, Apple Silicon

Fichiers clés

- `src/model.py` : implémentation Projection aléatoire
- `src/main.py` : pipeline d'apprentissage
- `scripts/plot_umap.py` : visualisation

Essayez vous-même

```
python src/main.py -epochs 30 -meta-paths AAA ACA ATA
```

Questions ?

Données d'entrée

- Auteurs : 28 702
- Conférences : 20
- Termes : 8 920

Matrices de relations

- A_{AA} : $28\,702 \times 28\,702$ (creuse)
- A_{AC} : $28\,702 \times 20$
- A_{AT} : $28\,702 \times 8\,920$
- A_{CA} : $20 \times 28\,702$
- A_{TA} : $8\,920 \times 28\,702$

Matrices des méta-chemins

- $M_{AAA}, M_{ACA}, M_{ATA}$: $28\,702 \times 28\,702$

Caractéristiques

- Chaque U_i : $28\,702 \times 256$

Répartition

- Matrices creuses : ≈ 50 Mo
- Tenseur de caractéristiques : $\approx 2,5$ Go
- Total : ≈ 3 Go (vs > 50 Go approche naïve)

Trucs d'implémentation

- SciPy pour toutes les matrices creuses
- Jamais de matérialisation de M^2
- Calcul itératif
- Cache disque pour les méta-chemins
- Accélération GPU hybride

figures/embedding.png